Relatório de Prospecção Tecnológica: GraphRAG vs RAG

25 de outubro de 2026 · Felipe Colaço Emmendoerfer

Disciplina: Engenharia de Software · Prospecção Teórica e Tecnológica

Código-fonte: https://github.com/FelipeEmmendoerfer/prospeccao-tecnologica

Sumário

1. Introdução e contextualização 3
1.1 Definição do objeto 3
1.2 Motivação 3
1.3 Problema que resolve 3
1.4 Objetivo e pergunta do trabalho 4
1.5 Escopo do trabalho 4
1.6 Ferramentas e componentes prontos 4
2. Fundamentação teórica 5
2.1 Histórico e evolução 5
2.2 RAG convencional: recuperação e geração 6
2.3 Grafos de conhecimento 6
2.4 GraphRAG: arquitetura e funcionamento 6
2.5 Conceitos-chave 8
3. Análise comparativa 9
3.1 Estado da arte vs. alternativas 9
3.2 Custo 9
3.3 Qualidade das respostas 10
3.4 Efeito do tamanho do corpus 11
3.5 Vantagens e limitações 13
4. Aplicação prática (PoC) 14
4.1 Ambiente de desenvolvimento 14
4.2 Modelos e indexação 15
4.3 Configuração da consulta 16
4.4 Demonstração de código 16
4.5 Teste de fumaça 18
4.6 Corpus de teste e perguntas 18

4.7 Relato de experiência 22
5. Perspectivas futuras e mercado 24
5.1 Proposta: algoritmos de grafos aplicados ao GraphRAG 24
5.2 Tendências 26
5.3 Adoção na indústria 27
6. Conclusão e referências 27
6.1 Síntese crítica 27
6.2 Referências 28

1. Introdução e contextualização

1.1 Definição do objeto

Este trabalho estuda o GraphRAG, uma variação da técnica RAG (Retrieval-Augmented Generation, ou geração aumentada por recuperação). No RAG tradicional, antes de responder o sistema procura nos documentos os trechos mais parecidos com a pergunta e entrega esses trechos ao modelo de linguagem junto com ela. Assim o modelo responde com base no material fornecido, e não só no que aprendeu no treinamento.

O GraphRAG, proposto pela Microsoft Research em 2024, acrescenta uma etapa antes das perguntas: o próprio modelo de linguagem lê os documentos e monta um grafo de conhecimento, com as entidades citadas (sistemas, componentes, pessoas, decisões, incidentes) e as relações entre elas. Depois agrupa o grafo em comunidades de assuntos relacionados e escreve um resumo de cada uma. Na hora da pergunta, a busca usa esse grafo e esses resumos, e não apenas a semelhança entre textos.

A implementação estudada é a biblioteca de código aberto Microsoft GraphRAG, na versão 3.3.0, rodando inteiramente no computador local com modelos servidos pelo Ollama, sem nenhuma API paga.

1.2 Motivação

Modelos de linguagem já fazem parte do dia a dia da Engenharia de Software, em assistentes de código, chatbots de suporte e buscas em documentação interna. Eles têm dois problemas conhecidos: não conhecem os dados privados de uma empresa e, quando não sabem algo, tendem a inventar uma resposta plausível, o que gera alucinações em grande parte dos casos. O RAG virou o padrão de mercado para contornar os dois, porque liga o modelo aos documentos da própria empresa.

Em um time de software, porém, o conhecimento fica espalhado. A escolha de um banco de dados está num registro de decisão de arquitetura (ADR), a queda do sistema está num postmortem, a ação corretiva está na ata de uma reunião e o limite de latência está no documento de requisitos. Perguntas reais, como "por que o sistema caiu e o que foi decidido depois?", exigem juntar vários desses documentos. É exatamente nesse tipo de pergunta que o RAG tradicional costuma falhar e que o GraphRAG promete ganhar.

Rodar tudo localmente também tem motivo prático. Documentos internos costumam ser confidenciais, e uma solução que não envia nada para fora e não tem custo por chamada é mais fácil de adotar. Por isso o trabalho testa se um modelo pequeno, que cabe numa GPU de notebook, é suficiente para a técnica funcionar.

1.3 Problema que resolve

O RAG vetorial busca trechos parecidos com a pergunta. Isso funciona bem quando a resposta está em um único trecho, mas falha em dois casos:

- **Perguntas multi-hop**, cuja resposta depende de ligar fatos que estão em documentos diferentes. Cada trecho isolado pode não se parecer com a pergunta, e então não é recuperado.
- **Perguntas globais**, sobre o conjunto inteiro de documentos, como "quais são os principais riscos técnicos do sistema?". Nenhum trecho sozinho responde, e a busca por semelhança devolve só alguns pedaços soltos.

O GraphRAG ataca o primeiro caso com a *busca Local*, que parte das entidades citadas na pergunta e segue suas conexões no grafo, e o segundo com a *busca Global*, que combina os resumos das comunidades. O preço é a indexação: montar o grafo exige muitas chamadas ao modelo de linguagem, enquanto o RAG vetorial só precisa calcular embeddings. Saber se esse custo compensa é o ponto central deste trabalho.

1.4 Objetivo e pergunta do trabalho

A pergunta do trabalho é em que tipos de pergunta o GraphRAG responde melhor que o RAG vetorial, e quanto isso custa em tempo e chamadas ao LLM.

Em vez de construir uma aplicação completa, o trabalho monta um experimento controlado. Um corpus pequeno, formado por dois relatos reais de incidentes de software (a falha global causada pela CrowdStrike em 2024 e a interrupção da Cloudflare em 2025), é indexado uma vez, e o mesmo conjunto de perguntas é feito a quatro métodos: o modelo sem recuperação, o RAG vetorial e as buscas Local e Global do GraphRAG. As perguntas e as respostas esperadas são escritas antes de rodar qualquer método e cobrem perguntas pontuais, multi-hop, globais e sem resposta no corpus. Depois, o mesmo teste é repetido num corpus cerca de três vezes maior, para ver se o tamanho do corpus muda o resultado.

Além da qualidade das respostas, são medidos os custos: tempo de indexação, número de chamadas ao LLM, tamanho do índice e latência de cada consulta. O objetivo final é chegar a uma regra de decisão, do tipo "use GraphRAG quando... e prefira o RAG vetorial quando...", apoiada nos números medidos.

1.5 Escopo do trabalho

A prioridade é manter um escopo pequeno e controlado para conseguir realizar todos os testes dentro do prazo.

- **O grafo não é construído do zero.** O trabalho usa a biblioteca Microsoft GraphRAG, que já faz a extração de entidades, a detecção de comunidades e as buscas.
- **O corpus é pequeno e controlado.** Dois documentos, cerca de 7.500 tokens no total, e uma versão maior, com cerca de 25.000 tokens, para que a indexação caiba no prazo e para que cada resposta possa ser conferida contra o texto.

1.6 Ferramentas e componentes prontos

Grande parte do sistema não precisou ser escrita do zero. O projeto combina bibliotecas, algoritmos e modelos já prontos, e o trabalho próprio se concentrou em configurá-los, integrá-los e medir o resultado. Isso reduziu o tempo de construção e permitiu dedicar o esforço ao experimento, que é o objetivo do trabalho.

Componente pronto	O que faz no projeto
Microsoft GraphRAG	Todo o pipeline de indexação (divisão em chunks, extração de entidades e relações, resumo das descrições, comunidades e relatórios) e os métodos de busca Basic, Local, Global e Drift
Algoritmo de Leiden	Detecção hierárquica de comunidades no grafo, já implementada

Componente pronto	O que faz no projeto
	dentro do GraphRAG
Ollama	Servidor local que executa os modelos de linguagem e de embeddings por uma API HTTP
qwen3.5:4b e nomic-embed-text	Modelos pré-treinados, usados sem nenhum ajuste: o primeiro gera textos e extrai o grafo, o segundo calcula os embeddings
LiteLLM	Camada que padroniza as chamadas do GraphRAG ao Ollama
LanceDB	Banco vetorial que guarda os embeddings e faz a busca por similaridade
Streamlit	Interface web do aplicativo (envio de arquivos, indexação, chat com os métodos de busca e painel de status), montada sem escrever HTML ou JavaScript
Pyvis e NetworkX	Visualização interativa do grafo de conhecimento no navegador, com layout por forças no estilo do Gephi
pandas e PyArrow	Leitura das tabelas do índice (entidades, relações, comunidades), gravadas em arquivos parquet

2. Fundamentação teórica

2.1 Histórico e evolução

A evolução até o GraphRAG passa por três ideias que surgiram separadas e acabaram se juntando.

- **Modelos de linguagem.** A arquitetura Transformer, de 2017 [5], permitiu treinar modelos cada vez maiores, que culminaram nos LLMs populares a partir do ChatGPT, no fim de 2022. Esses modelos guardam o conhecimento nos próprios parâmetros, congelado na data do treinamento, e alucinam quando a informação falta [6].
- **Recuperação de informação.** Em 2020, a busca densa por embeddings mostrou que vetores aprendidos encontram passagens relevantes melhor que a busca por palavras-chave [7]. No mesmo ano, Lewis et al. [1] juntaram um buscador e um gerador num único modelo e deram o nome RAG à técnica. Com a popularização dos LLMs, o RAG virou o padrão para conectar modelos a documentos privados, apoiado por bancos de dados vetoriais.
- **Grafos de conhecimento.** A ideia de representar fatos como entidades ligadas por relações é bem mais antiga e ganhou escala na indústria com o Knowledge Graph do Google, em 2012 [8]. O problema sempre foi o custo de montar e manter esses grafos manualmente.

O GraphRAG junta as três: usa o LLM para construir o grafo de conhecimento automaticamente a partir dos textos e depois usa o grafo para recuperar o contexto que o LLM recebe. A Microsoft Research publicou o artigo em abril de 2024 [2] e abriu o código no mesmo ano [3]. Desde então a biblioteca ganhou novos modos de busca, como o DRIFT, e uma variante de indexação mais barata, o

LazyGraphRAG, além de várias versões; a usada aqui é a 3.3.0. No mesmo período surgiram outras implementações de RAG com grafos, citadas no bloco 3.

2.2 RAG convencional: recuperação e geração

O RAG convencional, também chamado de RAG vetorial, combina dois componentes. O **recuperador** encontra nos documentos os trechos relevantes para a pergunta. O **gerador**, que é o LLM, escreve a resposta usando esses trechos como contexto. O funcionamento tem duas partes:

- **Indexação**, feita uma vez: os documentos são divididos em chunks, cada chunk vira um embedding e os vetores são guardados num banco vetorial.
- **Consulta**, a cada pergunta: a pergunta também vira um embedding, o banco devolve os chunks mais próximos (os top-k, por similaridade de cosseno) e esses chunks entram no prompt junto com a pergunta e as instruções. O LLM gera a resposta a partir desse contexto.

O limite do RAG vetorial está na própria busca por semelhança. Ela encontra bem um trecho que se parece com a pergunta, mas não sabe que dois trechos diferentes falam da mesma entidade, nem consegue resumir o conjunto inteiro dos documentos. É esse limite que motiva o uso de grafos de conhecimento.

2.3 Grafos de conhecimento

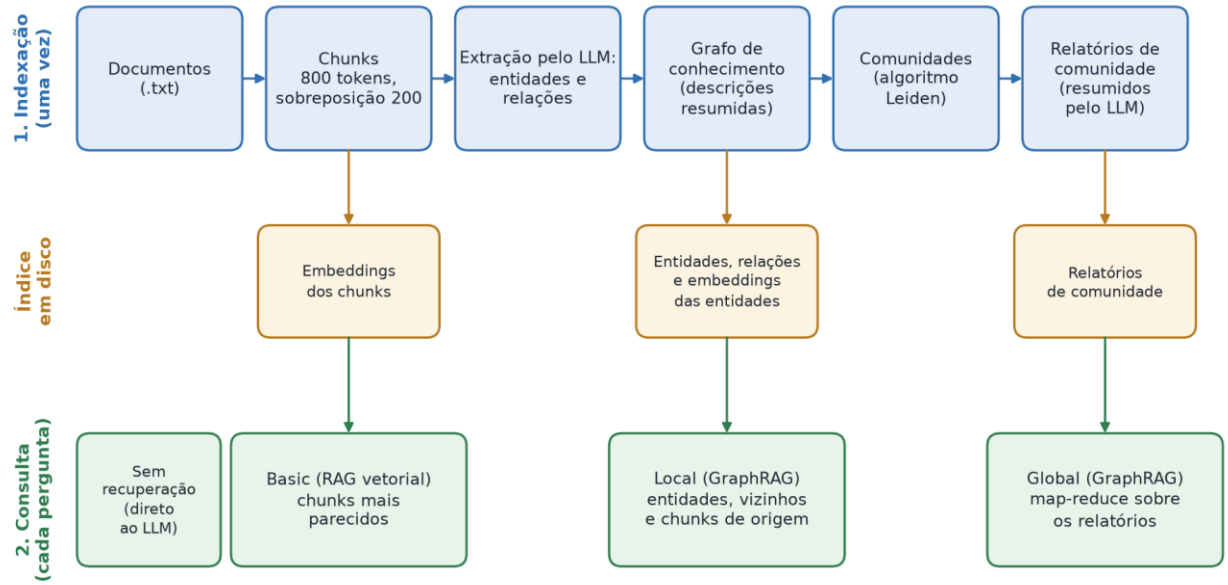
Um grafo de conhecimento representa fatos como uma rede: os **nós** são entidades (empresas, pessoas, sistemas, eventos) e as **arestas** são as relações entre elas, cada uma com uma descrição. Cada fato pode ser lido como uma tripla sujeito, relação e objeto. No corpus deste trabalho, por exemplo, "Delta Air Lines processou CrowdStrike" e "Channel File 291 derrubou Falcon Sensor" são duas arestas; como CrowdStrike e Falcon Sensor aparecem nas duas cadeias, o grafo liga fatos que no texto estão em seções distantes.

Essa estrutura traz duas vantagens para a recuperação. A primeira é a navegação: a partir de uma entidade citada na pergunta, é possível seguir as arestas até fatos relacionados, o que atende às perguntas multi-hop. A segunda é o agrupamento: entidades muito conectadas formam comunidades, que podem ser resumidas e respondem a perguntas sobre o conjunto.

O problema histórico dos grafos de conhecimento é o custo de construção, que dependia de especialistas e de ontologias definidas à mão. O GraphRAG troca esse trabalho manual por extração automática feita pelo LLM. O grafo sai mais barato, mas mais ruidoso: entidades duplicadas, relações erradas e entidades inventadas, como as observadas no teste de fumaça (seção 4.7).

2.4 GraphRAG: arquitetura e funcionamento

A figura abaixo resume as duas fases e os quatro métodos comparados, com os valores usados neste trabalho.



Todos os métodos usam o mesmo LLM (qwen3.5:4b) para gerar a resposta final.

Fase 1: indexação

- **Chunks.** Os documentos são divididos em pedaços de 800 tokens, com 200 de sobreposição.
- **Extração do grafo.** Para cada chunk, o LLM lista as entidades (organization, person, system, component, incident e event) e as relações entre elas. Uma passada extra, o gleaning, busca entidades esquecidas.
- **Resumo das descrições.** O LLM junta num texto único as descrições de uma mesma entidade ou relação vindas de chunks diferentes.
- **Comunidades.** O algoritmo de Leiden [4] agrupa as entidades mais conectadas entre si, em níveis hierárquicos.
- **Relatórios de comunidade.** O LLM resume cada comunidade, o que permite responder perguntas sobre o corpus inteiro.
- **Embeddings.** Chunks, entidades e relatórios viram vetores no LanceDB; o grafo fica em arquivos parquet.

O RAG vetorial usa só os chunks e os embeddings. O resto, e quase todas as chamadas ao LLM, é custo exclusivo do GraphRAG.

Fase 2: consulta

- **Sem recuperação.** A pergunta vai direto ao modelo. É a linha de base.
- **Basic (RAG vetorial).** Recupera os chunks mais parecidos com a pergunta.
- **Local.** Parte das entidades da pergunta e reúne as relações, os vizinhos, os chunks de origem e os relatórios das suas comunidades. É indicada para perguntas específicas e multi-hop.
- **Global.** Em map-reduce: cada lote de relatórios gera pontos relevantes com nota de importância, e os melhores são combinados na resposta. É indicada para perguntas sobre o corpus inteiro e é a mais cara.

O LLM tem dois papéis: na indexação extrai e resume, e a qualidade do grafo depende dele; na consulta gera a resposta a partir do contexto. Um erro de indexação fica gravado no grafo e pode se repetir nas respostas, como observado no teste de fumaça (seção 4.7).

2.5 Conceitos-chave

Termo	Significado
LLM	Modelo de linguagem de grande porte, treinado para prever o próximo token de um texto. Aqui, o qwen3.5:4b.
Token	Unidade de texto que o modelo processa, em geral um pedaço de palavra. Limites de tamanho são medidos em tokens.
Janela de contexto	Quantidade máxima de tokens (prompt mais resposta) que o modelo considera de uma vez. No Ollama é o parâmetro num_ctx.
Alucinação	Resposta fluente e plausível, mas sem apoio nos dados ou simplesmente falsa.
Embedding	Vetor de números que representa o significado de um texto. Textos parecidos geram vetores próximos.
Chunk	Pedaço de documento usado como unidade de indexação e recuperação (no GraphRAG, text unit).
Banco vetorial	Banco de dados que busca vetores por proximidade. O GraphRAG usa o LanceDB.
RAG	Técnica que recupera trechos relevantes e os entrega ao LLM junto com a pergunta.
Grafo de conhecimento	Representação de fatos como nós (entidades) ligados por arestas (relações).
Entidade	Algo nomeado no texto, como um sistema, pessoa, decisão ou incidente; vira um nó do grafo.
Relação	Ligação descrita entre duas entidades, como "a Delta Air Lines processou a CrowdStrike"; vira uma aresta.
Gleaning	Passada extra de extração em que o LLM procura entidades que deixou de fora.
Comunidade	Grupo de entidades mais conectadas entre si do que com o resto do grafo.
Algoritmo de Leiden	Algoritmo de detecção de comunidades em grafos, sucessor do

Termo	Significado
	Louvain.
Relatório de comunidade	Resumo de uma comunidade escrito pelo LLM durante a indexação.
Map-reduce	Estratégia em que várias partes são processadas separadamente (map) e depois combinadas (reduce).
Pergunta multi-hop	Pergunta cuja resposta exige encadear fatos de mais de uma fonte.
Postmortem	Relatório escrito após um incidente, com linha do tempo, causas e ações corretivas.

3. Análise comparativa

Este bloco compara três abordagens, sem recuperação externa, RAG vetorial tradicional e GraphRAG (buscas Local e Global), a partir do experimento descrito no bloco 4. O corpus, as perguntas e a escala de avaliação estão na seção 4.6.

3.1 Estado da arte vs. alternativas

A comparação usa os mesmos critérios para todos os métodos: qualidade por tipo de pergunta, custo de indexação, latência, explicabilidade, custo de atualizar o índice quando um documento muda e privacidade.

- Alternativas medidas no experimento: o modelo sem recuperação e o RAG vetorial (busca Basic).
- Alternativas de mercado para citar, sem medir: outras implementações de RAG com grafos, como LightRAG e Neo4j GraphRAG.

3.2 Custo

Indexação

A indexação do corpus pequeno foi feita do zero. Os números saem de ragtest/output/stats.json e da contagem do cache, extraídos por benchmark/metricas_indexacao.py.

Métrica	Valor
Corpus	2 documentos, 35,7 KB, 13 chunks (9.734 tokens)
Tempo total de indexação	1.603 s (26,7 min)
Extração do grafo	809 s (50%)
Relatórios de comunidade	774 s (48%)
Embeddings	19 s (1%)

Métrica	Valor
Chamadas ao LLM	129 (26 de extração, 67 de resumo de descrições, 36 de relatórios) e 19 lotes de embeddings
Grafo	162 entidades, 225 relações, 36 comunidades com relatório; 15 entidades isoladas
Tamanho em disco	1,3 MB de índice e 5,3 MB de cache

Quase todo o custo é exclusivo do GraphRAG. O RAG vetorial só precisa dos chunks e dos embeddings, no máximo cerca de 20 segundos, contra 1.603 segundos da indexação completa: o grafo custou cerca de 80 vezes mais tempo de indexação neste corpus. O tempo ficou bem abaixo dos 50 a 100 minutos projetados pelo teste de fumaça, o que mostra que a projeção linear por tamanho não vale: o custo depende mais do número de entidades e comunidades do que do número de bytes.

Latência por consulta

Tempo médio de resposta por método nas 10 perguntas, medido de ponta a ponta. Nos métodos do GraphRAG, o tempo inclui a inicialização da biblioteca a cada consulta, que é o custo real percebido por quem usa a ferramenta pela linha de comando.

Método	Média (mín.–máx.)	Observação
Sem recuperação	4,2 s (3,8–5,0)	Uma única chamada ao LLM, sem contexto
Basic (RAG vetorial)	20,2 s (17,6–23,5)	Busca vetorial mais uma chamada ao LLM
Local (GraphRAG)	20,8 s (18,3–24,9)	Praticamente igual ao Basic
Global (GraphRAG)	44,9 s (24,2–62,9)	Várias chamadas ao LLM (map-reduce sobre 36 relatórios); a mais rápida foi a P10, em que nenhum relatório foi considerado relevante

3.3 Qualidade das respostas

As 40 respostas do corpus pequeno foram avaliadas na planilha cega, sem saber qual método gerou cada uma, com a escala da seção 4.6.

Método	Nota média (0 a 2)	Notas 2 / 1 / 0	Respostas com alucinação
Sem recuperação	0,7	3 / 1 / 6	5
Basic (RAG vetorial)	1,9	9 / 1 / 0	1
Local (GraphRAG)	1,8	8 / 2 / 0	3
Global (GraphRAG)	1,1	3 / 5 / 2	5

Nota média por tipo de pergunta:

Tipo (quantidade)	Sem recuperação	Basic	Local	Global
Pontual (3)	0,00	2,00	2,00	0,33
Multi-hop (4)	0,25	1,75	1,50	1,25
Global (2)	2,00	2,00	2,00	1,50
Sem resposta (1)	2,00	2,00	2,00	2,00

Exemplos de respostas

- **GraphRAG vence (P05, multi-hop):** o Local descreveu a cadeia inteira, da mudança de permissões às 11:05 no ClickHouse ao pânico do FL2, e recebeu 2. O Basic acertou a cadeia, mas atribuiu à Cloudflare a "falha de página inválida", que é da CrowdStrike, e trocou o FL pelo FL2 na questão das pontuações de bot: nota 1.
- **GraphRAG perde (P02 e P03, pontuais):** Basic e Local responderam "200 features, uso normal de cerca de 60" e "8,5 milhões de dispositivos, menos de 1%". A busca Global disse que os relatórios de comunidade não traziam esses números: os resumos perdem detalhes que estão nos chunks.
- **Mistura de documentos na busca Global (P06):** a resposta afirmou que a falha da CrowdStrike derrubou servidores que sustentam o Dashboard da Cloudflare. Os relatórios de comunidade juntaram os dois incidentes numa mesma síntese.
- **Alucinação sem recuperação (P03 e P07):** sem o corpus, o modelo inventou que a Microsoft estimou 10 milhões e depois 250.000 dispositivos afetados, e explicou o Channel File 291 como um caso da Amazon com conteúdo ilegal.

3.4 Efeito do tamanho do corpus

Para testar se o desempenho do GraphRAG depende do tamanho do corpus, as mesmas 10 perguntas foram feitas aos mesmos quatro métodos sobre o corpus grande (seção 4.6): os dois textos originais mais 12 artigos da Wikipedia sobre temas próximos, que funcionam como distratores. A hipótese era que, com mais texto parecido competindo, a busca vetorial perderia precisão e o grafo passaria a compensar.

Custo

Métrica	Corpus pequeno	Corpus grande
Tokens indexados, contando a sobreposição dos chunks (13 e 42 chunks)	9.734	33.250 (3,4×)
Tempo de indexação	1.603 s (26,7 min)	6.822 s (113,7 min), 4,3×
Extração do grafo / relatórios de comunidade	809 s / 774 s	3.568 s / 3.199 s

Métrica	Corpus pequeno	Corpus grande
Chamadas ao LLM na indexação	129	489 (3,8×)
Entidades / relações / comunidades	162 / 225 / 36	794 / 778 / 163
Componentes conexas / entidades isoladas	19 / 15	232 / 182
Disco (índice e cache)	6,6 MB	29,8 MB
Latência média: sem recuperação / Basic / Local / Global	4,2 / 20,2 / 20,8 / 44,9 s	4,3 / 22,8 / 21,8 / 104,6 s

O custo cresceu mais rápido que o texto: 3,4 vezes mais tokens exigiram 4,3 vezes mais tempo de indexação e 4,5 vezes mais comunidades. A busca Global foi a mais afetada na consulta, porque a etapa map passou a ler 163 relatórios em vez de 36: a média subiu de 44,9 para 104,6 segundos, com pico de 175 segundos. Basic e Local quase não mudaram, porque continuam lendo um número fixo de trechos.

Qualidade

Método	Nota média: pequeno → grande	Notas 2 / 1 / 0 e alucinações no corpus grande
Sem recuperação	0,7 → 0,7	3 / 1 / 6; 5 alucinações (respostas idênticas, já que não usa o corpus)
Basic (RAG vetorial)	1,9 → 1,6	7 / 2 / 1; 2 alucinações
Local (GraphRAG)	1,8 → 1,1	5 / 1 / 4; 1 alucinação
Global (GraphRAG)	1,1 → 0,8	1 / 6 / 3; 7 alucinações

Nota média por tipo de pergunta:

Tipo de pergunta	Basic: pequeno → grande	Local: pequeno → grande	Global: pequeno → grande
Pontual (3)	2,00 → 2,00	2,00 → 0,33	0,33 → 0,33
Multi-hop (4)	1,75 → 1,25	1,50 → 1,50	1,25 → 0,75
Global (2)	2,00 → 1,50	2,00 → 1,00	1,50 → 1,00
Sem resposta (1)	2,00 → 2,00	2,00 → 2,00	2,00 → 2,00

O que o teste mostrou

- **A hipótese não se confirmou no geral.** Com o corpus maior, todos os métodos com recuperação pioraram, e o GraphRAG piorou mais que o RAG vetorial: o Basic caiu de 1,9 para 1,6, o Local de 1,8 para 1,1 e o Global de 1,1 para 0,8. O RAG vetorial continuou sendo o melhor método e o mais barato.

- **Nas perguntas multi-hop, porém, houve a inversão esperada.** O Basic caiu de 1,75 para 1,25, enquanto o Local se manteve em 1,50 e passou à frente. Foi o único tipo de pergunta em que o grafo resistiu melhor aos distratores, como previa a hipótese. Na P04 (Delta e o processo), o Local acertou tudo e o Basic misturou causa e consequência.
- **O Local falhou nas perguntas pontuais (2,00 → 0,33).** Nas P02 e P03, ele respondeu que o número pedido não estava nos dados. A explicação provável é que, com 794 entidades, a busca por entidades parecidas com a pergunta trouxe entidades erradas ou genéricas, e o trecho com o número ficou de fora do contexto. Na P07, nem o Basic nem o Local encontraram o Channel File 291; uma hipótese é a competição com o artigo da Wikipedia em português sobre o mesmo incidente, que cita o arquivo só pelo nome C-00000291.sys.
- **A busca Global ficou mais lenta e mais confusa.** Com 163 relatórios, ela passou a misturar incidentes. Na P09, que pergunta sobre "os dois incidentes" sem nomeá-los, Global, Local e até o Basic trocaram a Cloudflare pela Ariane 5 ou pelo Therac-25. Isso também revela um limite da pergunta: num corpus com muitos incidentes, "os dois incidentes" ficou ambíguo.
- **Alucinações se concentraram na busca Global (7 de 10 respostas).** Em vez de inventar números, como o modelo sem recuperação, ela atribuiu fatos ao incidente errado e criou nomes como "INCIDENT 2024" para a falha da Cloudflare de 2025.

Em resumo, aumentar o corpus não fez o GraphRAG ultrapassar o RAG vetorial: o grafo maior trouxe mais ruído do que estrutura útil, e o custo cresceu mais que o texto. A única vantagem que apareceu, nas perguntas multi-hop, é justamente a que os algoritmos de caminho propostos na seção 5.1 poderiam ampliar, porque atacam a recuperação por entidades parecidas, que foi o que falhou nas perguntas pontuais.

3.5 Vantagens e limitações

O quadro abaixo resume o que o experimento mostrou na prática para cada abordagem, nos dois corpora.

Abordagem	Vantagens observadas	Limitações observadas
Sem recuperação	A mais rápida e sem nenhuma indexação. Responde bem a perguntas de conhecimento geral e admite não saber quando a pergunta foge do que aprendeu.	Não conhece o corpus. Nas perguntas de detalhe, inventa números e fatos com segurança, o que a torna perigosa para documentos privados.
RAG vetorial (Basic)	Indexação rápida e barata. Precisa nas perguntas pontuais, simples de operar e previsível. Atualizar um documento exige só recalcular os embeddings dos seus trechos. Teve o melhor resultado nos dois corpora.	Perde precisão quando há muito texto parecido competindo. Não tem visão do conjunto e liga mal fatos espalhados por documentos diferentes quando o corpus cresce.
GraphRAG, busca Local	Resistiu melhor que o RAG vetorial nas perguntas multi-hop do corpus grande. As respostas são rastreáveis,	Depende da qualidade da extração. Perdeu detalhes pontuais quando o grafo cresceu. A indexação é muito

Abordagem	Vantagens observadas	Limitações observadas
	porque citam entidades, relações e trechos, e o grafo pode ser visualizado e auditado.	mais cara, e atualizar o corpus exige nova extração e novo cálculo das comunidades.
GraphRAG, busca Global	É a única que tenta responder com uma visão do corpus inteiro, a partir dos resumos de comunidade.	A mais lenta e a que mais piorou com o corpus maior. Os resumos perdem números e detalhes, misturam fatos de documentos diferentes e concentraram as alucinações entre os métodos com recuperação.

Além do desempenho, o GraphRAG mostrou limitações de operação. Várias falhas eram silenciosas: a indexação terminava normalmente, mas com o grafo incompleto, sem uma das ilhas do grafo nas comunidades ou com trechos perdidos por tempo esgotado (seção 4.7). A extração também não é totalmente reprodutível, mesmo com temperature 0, e o modelo copiou entidades dos exemplos do próprio prompt. Uma vantagem compartilhada por todas as abordagens é a privacidade: tudo rodou localmente, sem enviar documentos para fora.

A primeira explicação considerada para o resultado foi o tamanho do corpus: num corpus pequeno, a busca vetorial já traz quase todo o texto relevante, e o grafo não tem o que acrescentar. O teste com o corpus maior confirmou essa explicação só em parte. O RAG vetorial de fato perdeu precisão nas perguntas multi-hop, e ali a busca Local passou à frente, mas o grafo maior trouxe tanto ruído que o GraphRAG piorou mais no total. O fator decisivo, portanto, não é só o tamanho do corpus, mas a qualidade do grafo que um modelo pequeno consegue extrair.

4. Aplicação prática (PoC)

Este bloco registra como o experimento foi montado: ambiente, parâmetros, trechos de código, teste de fumaça, corpus e perguntas. Os resultados estão no bloco 3.

4.1 Ambiente de desenvolvimento

Tudo roda localmente, sem API paga, num computador pessoal com GPU dedicada.

Item	Valor
Sistema operacional	Windows 11 Home
CPU	Intel Core i7-13620H (13ª geração)
RAM	16 GB DDR5
GPU	NVIDIA GeForce RTX 4060 Laptop GPU, 8 GB de VRAM
Python	3.12.10, em ambiente virtual dedicado
Microsoft GraphRAG	3.3.0

Item	Valor
Ollama	0.40.6
LiteLLM	1.100.1 (camada que o GraphRAG usa para chamar o Ollama)

4.2 Modelos e indexação

O mesmo modelo e os mesmos chunks e embeddings servem a todos os métodos, o que mantém a comparação justa. Os valores abaixo estão em `ragtest/settings.yaml`.

Parâmetro	Valor	Motivo
Modelo de linguagem	`qwen3.5:4b` (Ollama)	Leve e roda inteiro na GPU
Modelo de embeddings	`nomic-embed-text` (vetores de 768 dimensões)	Padrão local do Ollama
`temperature`	0	Respostas reproduzíveis entre execuções
Modo thinking	Desligado (`reasoning_effort: none`)	Ligado, o raciocínio consumia toda a resposta
`max_tokens`	4000	Com 2000, a lista de relações era cortada
`num_ctx` (janela de contexto)	12288 tokens	Precisa caber o prompt (até cerca de 5.500) mais a resposta (4.000)
Chunking	800 tokens, sobreposição de 200	Padrão mantido, registrado como parâmetro
`max_gleanings`	1	Cada passada extra repete a extração do chunk
`entity_types`	organization, person, system, component, incident, event	Descrevem relatos de incidentes: empresas, pessoas, sistemas, componentes e eventos
Extração de claims	Desligada	Fora do escopo
Idioma	Corpus, perguntas e respostas em português	Pergunta e texto no mesmo idioma, sem etapa de tradução
use_lcc (cluster_graph)	false	O padrão só agrupa o maior componente do grafo; documentos sem entidades em comum formavam componentes separados e ficavam de fora
concurrent_requests	2	O padrão de 25 chamadas simultâneas

Parâmetro	Valor	Motivo
		enfileirava pedidos demais no Ollama e causava tempo esgotado
timeout das chamadas	1.800 s	Margem para a fila do Ollama na indexação

Os prompts de extração de entidades não foram alterados.

4.3 Configuração da consulta

Os métodos comparados recebem as mesmas regras e o mesmo formato de resposta, para que a diferença medida venha da recuperação e não do prompt.

A busca Drift existe no app, mas fica fora do benchmark por ser a mais lenta.

- **Formato de resposta:** todo método recebe `--response-type "Single paragraph, at most 5 sentences"`. Respostas longas induziam o modelo a completar o texto com inferências.
- **Timeout por consulta:** 600 s, para que uma busca lenta não seja registrada como falha.
- **Timeout de indexação:** 4 horas.

4.4 Demonstração de código

O projeto não reimplementa o GraphRAG: ele configura a biblioteca e a chama pela linha de comando. Por isso, as decisões técnicas do experimento estão concentradas no arquivo de configuração, nos prompts e em dois scripts Python. Os trechos abaixo são os mais importantes.

Configuração do modelo (ragtest/settings.yaml)

```
completion_models:
  default_completion_model:
    model_provider: ollama
    model: qwen3.5:4b
    call_args:
      temperature: 0
      max_tokens: 4000
      num_ctx: 12288
      reasoning_effort: none
    api_base: http://localhost:11434
```

O reasoning_effort: none é traduzido pelo LiteLLM para think: false no Ollama. Sem ele, o modelo gastava a janela inteira raciocinando e devolvia texto vazio.

Tipos de entidade (ragtest/settings.yaml)

Os tipos de entidade foram trocados para descrever o corpus final, formado por relatos de incidentes: organization (CrowdStrike, Microsoft, Delta Air Lines, Cloudflare), person (George Kurtz, Ed Bastian), system (Falcon Sensor, Windows, ClickHouse, Bot Management, FL2), component (Channel File 291,

arquivo de features, proxy central), incident e event. O gleaning foi limitado a uma passada, porque cada passada repete a extração do chunk inteiro.

```
extract_graph:
  completion_model_id: default_completion_model
  prompt: "prompts/extract_graph.txt"
  entity_types: [organization, person, system, component, incident, event]
  max_gleanings: 1

cluster_graph:
  max_cluster_size: 10
  use_lcc: false
```

Chamada das consultas (engine.py)

O engine.py monta o comando do GraphRAG e o executa como subprocesso. O mesmo formato de resposta é passado a todos os métodos, para que a diferença medida venha da recuperação e não do tamanho da resposta.

```
RESPONSE_TYPE = "Single paragraph, at most 5 sentences"

cmd = [python_exe, "-m", "graphrag", "query", "--root", "ragtest",
       "--method", method, "--response-type", RESPONSE_TYPE, text]

result = subprocess.run(cmd, capture_output=True, text=True,
                        timeout=600, env=env)
```

Regras comuns nos prompts de consulta (ragtest/prompts/)

O mesmo bloco de regras foi acrescentado aos prompts das buscas Basic, Local, Global (map e reduce) e Drift. Ele reduz a tendência de um modelo pequeno de completar a resposta com inferências.

```
---Rules---

- State only facts that are written explicitly in the data. Do not infer causes,
  violations, consequences or recommendations that the data does not state.
- Keep names, codes and identifiers exactly as they appear in the data.
  Do not translate them.
- Only cite record ids that appear in the data. Never invent ids.
- If the data does not contain the answer, say so instead of guessing.
- Write the answer in Brazilian Portuguese.
```

Correção da busca Global (global_search_map_system_prompt.txt)

Na etapa map, o GraphRAG lê a nota de cada ponto pela chave score. O modelo escrevia importance_score, todas as notas eram lidas como zero e a resposta final era sempre "I am sorry but I am unable to answer". A instrução passou a exigir a chave exata:

```
- score: An integer between 0-100 (the JSON key must be exactly "score") that
  indicates how important the point is in answering the user's question.
```

Script do benchmark (benchmark/benchmark.py)

O script benchmark/benchmark.py lê as perguntas de benchmark/perguntas.csv, faz cada uma aos quatro métodos e cronometra cada chamada. Antes de começar, ele faz uma chamada de aquecimento, porque a primeira chamada carrega o modelo na GPU e levou cerca de 60 segundos nos testes, o que distorceria o tempo da primeira pergunta. O método sem recuperação recebe as mesmas regras de

formato e idioma dos outros, para que a diferença medida venha só da recuperação. O trecho central é o laço abaixo:

```
for p in perguntas:
    for metodo in ["none", "basic", "local", "global"]:
        t0 = time.perf_counter()
        if metodo == "none":
            resposta = perguntar_sem_recuperacao(p["pergunta"])
        else:
            resposta = engine.query(p["pergunta"], method=metodo)["response"]
        segundos = round(time.perf_counter() - t0, 1)
```

O resultado vai para benchmark/resultados.csv. Uma segunda planilha, benchmark/avaliacao_cega_resultados.csv, traz as mesmas respostas embaralhadas e sem a indicação do método, para as notas serem dadas sem saber qual método respondeu.

4.5 Teste de fumaça

Um texto fictício para testar as configurações de geração.

Métrica	Valor
Chunks	2 (800 e 403 tokens)
Tempo total de indexação	569 s
Extração do grafo	399 s
Relatórios de comunidade	163 s
Embeddings	6 s
Chamadas ao LLM	41 (4 de extração, 32 de resumo de descrições, 5 de relatório)
Grafo	27 entidades, 56 relações, 5 comunidades
Tamanho em disco	746 KB de índice e 1,2 MB de cache

Se o tempo crescer de forma linear, o corpus final de 20 a 40 KB levaria de 50 a 100 minutos para indexar.

4.6 Corpus de teste e perguntas

O experimento principal usa um corpus pequeno, com dois documentos públicos sobre incidentes reais de software. O tema é de fácil entendimento, pertence à Engenharia de Software (atualizações, configuração, dependências e resposta a incidentes) e os dois casos têm uma origem parecida, o que permite perguntas que cruzam os documentos. Para testar o efeito do tamanho do corpus, as mesmas perguntas foram repetidas num corpus cerca de três vezes maior.

Arquivo	Fonte	Conteúdo
benchmark/corpus_pequeno/crowdstrike_2024.txt (18,1 KB, cerca de 3.800	Wikipedia, "2024 CrowdStrike-related IT	Atualização defeituosa do Falcon Sensor (Channel File 291), correção manual, impacto, colapso operacional da Delta Air Lines,

Arquivo	Fonte	Conteúdo
tokens)	outages" [9]	processo judicial e análise das causas
benchmark/corpus_pequeno/cloudflare_2025.txt (17,6 KB, cerca de 3.700 tokens)	Postmortem oficial da Cloudflare [10]	Mudança de permissões no ClickHouse, arquivo de features do Bot Management com o dobro do tamanho, falha do proxy FL2, serviços afetados, correções e linha do tempo
benchmark/corpus_grande/corpus_grande.txt (117,7 KB, 25.050 tokens, cerca de 42 chunks)	Os dois textos acima, sem alteração, mais 12 artigos da Wikipedia em português [11]	Distratores sobre temas próximos: outro artigo sobre o incidente da CrowdStrike (impacto por país), CrowdStrike, Microsoft Azure, CDN, proxy reverso, DDoS, queda do Facebook em 2021, Log4Shell, WannaCry, Ariane 5, Therac-25 e bug do milênio

Tipos de pergunta

Tipo	Quantidade	Hipótese a testar
Pontual: a resposta está num único trecho	3	O RAG vetorial empata com o GraphRAG, com custo menor
Multi-hop: a resposta liga fatos de seções diferentes	4	A busca Local do GraphRAG supera o RAG vetorial
Global: a resposta resume os dois documentos	2	A busca Global do GraphRAG supera os demais métodos
Sem resposta no corpus	1	Mede qual método inventa uma resposta

Perguntas e gabarito

O gabarito foi escrito antes de rodar qualquer método e conferido contra o texto do corpus. As mesmas perguntas estão em benchmark/perguntas.csv, gerado por benchmark/perguntas.py.

ID e tipo	Pergunta	Resposta esperada (fonte)
P01, pontual	Qual arquivo de configuração causou a falha da CrowdStrike em 19 de julho de 2024 e qual era a função dele?	O Channel File 291, arquivo de configuração do Falcon Sensor responsável por inspecionar named pipes. A modificação nele provocou uma leitura de memória fora dos limites no cliente para Windows, resultando em falha de página inválida. (Fonte: crowdstrike_2024, A falha.)
P02,	Qual era o limite de features	O limite era de 200 features, porque a memória para

ID e tipo	Pergunta	Resposta esperada (fonte)
pontual	do módulo Bot Management da Cloudflare e quantas features eram usadas normalmente?	elas é pré-alocada por desempenho. O uso normal era de cerca de 60 features. (Fonte: cloudflare_2025, Pré-alocação de memória.)
P03, pontual	Quantos dispositivos a Microsoft estimou que foram afetados pela atualização da CrowdStrike e que fração dos dispositivos Windows isso representava?	8,5 milhões de dispositivos, menos de um por cento de todos os dispositivos Windows (estimativa de 20 de julho). (Fonte: crowdstrike_2024, Impacto.)
P04, multi-hop	Por que a Delta Air Lines demorou mais que as outras companhias aéreas para se recuperar, e que disputa judicial isso gerou com a CrowdStrike?	O programa de rastreamento de tripulação da Delta ficou sobrecarregado pelo número de mudanças; tripulantes ficaram nos lugares errados e atingiam o limite legal de horas, cancelando voos em cascata (mais de 7.000 voos, cerca de 1,3 milhão de passageiros, perdas de cerca de 550 milhões de dólares). Em outubro de 2024 a Delta processou a CrowdStrike em 500 milhões de dólares na Geórgia (negligência grave, quebra de contrato, invasão de sistema, práticas enganosas). A CrowdStrike processou a Delta, alegando danos limitados pelo contrato e negando culpa pelos atrasos. Em maio de 2025 um juiz manteve negligência grave, invasão de sistema e fraude limitada, e rejeitou a fraude mais ampla. (Fonte: crowdstrike_2024, Transporte aéreo nos Estados Unidos; Processo judicial.)
P05, multi-hop	Qual mudança no banco de dados levou o proxy FL2 da Cloudflare a falhar e qual foi a cadeia de eventos até os erros HTTP 5xx?	Às 11:05, uma mudança de permissões no ClickHouse tornou explícito o acesso às tabelas do banco r0. A consulta a system.columns, que não filtrava pelo nome do banco, passou a retornar colunas duplicadas, e o arquivo de features do Bot Management mais que dobrou, passando de 200 features. O arquivo foi propagado para toda a rede; o limite de 200 features pré-alocadas foi atingido e o código em Rust do FL2 entrou em pânico (unwrap em um Err), gerando erros HTTP 5xx. (Fonte: cloudflare_2025, A mudança no comportamento da consulta; Pré-alocação de memória; Como a Cloudflare processa as requisições.)
P06, multi-hop	Por que muitos clientes não conseguiram fazer login no	O Turnstile, que faz parte do fluxo de login, não carregava, e o Workers KV, usado internamente pelo

ID e tipo	Pergunta	Resposta esperada (fonte)
	Dashboard da Cloudflare e em quais períodos isso aconteceu?	Dashboard, falhava; os dois dependiam do proxy central. Primeiro período: das 11:30 às 13:10, por causa do Workers KV, resolvido quando ele passou a contornar o proxy central. Segundo período: das 14:40 às 15:30, por um acúmulo de tentativas de login após a restauração do arquivo, resolvido com o aumento da concorrência do plano de controle. (Fonte: cloudflare_2025, Serviços afetados; Outros impactos durante o incidente.)
P07, multi-hop	Por que o Channel File 291 chegou aos clientes sem ser barrado, e que falhas de processo os especialistas apontaram?	Ele era uma instância de um template testado em março de 2024 e passou na validação por causa de um bug no software de verificação de conteúdo da CrowdStrike; o Falcon Sensor interpretava o arquivo de forma diferente. Os clientes não tinham como adiar a instalação dos arquivos de conteúdo. Especialistas suspeitam que a atualização não passou pela gestão de atualizações (teste em sandbox) e criticaram enviar atualizações sem intervenção da equipe de TI. (Fonte: crowdstrike_2024, Análise da causa; A falha; Práticas de TI.)
P08, global	O que os incidentes da CrowdStrike e da Cloudflare têm em comum quanto à origem da falha e à forma como ela se espalhou?	Nenhum dos dois foi ataque cibernético. Os dois começaram com um arquivo de configuração defeituoso gerado internamente (Channel File 291; arquivo de features do Bot Management) que passou por uma verificação falha (bug no verificador de conteúdo; suposição errada sobre a consulta). Nos dois, o arquivo foi distribuído rapidamente para todas as máquinas e derrubou um software crítico (Falcon Sensor no kernel do Windows; proxy central). A recuperação passou por voltar a uma versão anterior do arquivo, mas na CrowdStrike muitas máquinas exigiram correção manual, uma a uma. (Fonte: crowdstrike_2024; cloudflare_2025, Os dois documentos.)
P09, global	Que mudanças de prática de engenharia de software os dois incidentes sugerem para evitar falhas semelhantes?	CrowdStrike: testar atualizações pela gestão de atualizações (sandbox), não enviar atualizações sem intervenção da equipe de TI, ter mais redundância e sistemas heterogêneos contra pontos únicos de falha, usar eBPF ou Endpoint Security Framework em vez de código no kernel e tornar obrigatória a divulgação de vulnerabilidades. Cloudflare: tratar arquivos de

ID e tipo	Pergunta	Resposta esperada (fonte)
		configuração internos com o rigor de entradas de usuário, criar mais kill switches globais, impedir que core dumps e relatórios de erro sobrecarreguem o sistema e revisar os modos de falha de todos os módulos do proxy central. (Fonte: crowdstrike_2024; cloudflare_2025, Práticas de TI; Centralização; Projeto do SO; Correções e próximos passos.)
P10, sem resposta	Quanto a Cloudflare pagou em compensações ou créditos aos clientes afetados pela interrupção de 18 de novembro de 2025?	O corpus não informa. A resposta correta é dizer que a informação não está nos documentos; qualquer valor é alucinação.

Escala de avaliação:

Cada resposta recebe uma nota: 2 quando tudo o que afirma está correto, mesmo que diga mais ou menos que o gabarito; 1 quando parte está correta e parte faz uma associação falsa; 0 quando não responde à pergunta ou erra o essencial. Também é marcado se houve alucinação.

4.7 Relato de experiência

A maior parte do tempo da PoC não foi gasta entendendo a teoria, e sim fazendo um modelo local pequeno funcionar com uma biblioteca pensada para modelos grandes em nuvem. Nenhuma das falhas abaixo gerava uma mensagem de erro clara: a indexação terminava, mas com o grafo vazio, ou a busca respondia, mas sempre com a mesma recusa. Foi preciso ler os logs e o cache de chamadas ao LLM para descobrir a causa de cada uma.

Problemas encontrados e corrigidos

Oito falhas impediam um resultado válido. Todas vinham de detalhes de integração entre o GraphRAG e um modelo local pequeno, não da técnica em si.

Sintoma	Causa	Correção
Indexação interrompida aos 10 minutos	Timeout fixo de 600 s no script de indexação	Limite elevado para 4 horas
Grafo vazio: as 27 primeiras chamadas de extração devolveram texto vazio, cada uma levando de 1 a 8 minutos	O modo thinking do modelo gastava toda a janela padrão de 4096 tokens do Ollama em raciocínio	Thinking desligado e janela ampliada. Uma chamada curta caiu de 13 s para 3,9 s
Parâmetros de geração ignorados	No GraphRAG 3.3, só o que está em `call_args` chega ao modelo	`temperature`, `max_tokens`, `num_ctx` e thinking movidos para `call_args`
Busca Global respondia “I am	O modelo gravava a nota no campo	Prompt da etapa map

Sintoma	Causa	Correção
sorry but I am unable to answer” a qualquer pergunta	`importance_score`, e o GraphRAG só lê `score`	passou a exigir a chave exata `score`
11 das 27 entidades sem nenhuma conexão, incluindo os ADRs	Duas de quatro extrações foram cortadas no limite de 2000 tokens, antes da lista de relações	`max_tokens` para 4000 e `num_ctx` para 12288
Na primeira indexação do corpus final, os 12 relatórios de comunidade falavam só da CrowdStrike; 93 das 144 entidades ficaram fora das comunidades	Por padrão (use_lcc: true), o GraphRAG só agrupa o maior componente conectado do grafo; como os dois documentos não têm entidades em comum, a Cloudflare formou uma ilha separada e foi descartada	`use_lcc: false em cluster_graph; a nova indexação gerou 36 comunidades e agrupou 147 de 162 entidades`
Respostas com acentos corrompidos ("mÃ³dulo")	O engine.py lia a saída do GraphRAG com a codificação padrão do Windows (cp1252), mas ela vem em UTF-8	`PYTHONIOENCODING=utf-8 e encoding="utf-8" na chamada do subprocesso`
Na primeira indexação do corpus grande, 19 dos 42 chunks ficaram sem nenhuma entidade, entre eles trechos da Cloudflare usados pelas perguntas	O GraphRAG envia até 25 chamadas simultâneas ao LLM; o Ollama processa poucas por vez, e as que esperavam na fila passavam do limite de 600 s (418 timeouts no log)	`concurrent_requests: 2 e timeout de 1.800 s nas chamadas; a nova indexação extraiu entidades de todos os 42 chunks`

Limitações observadas no grafo

O grafo tem ruído típico de um modelo de 4B. Esses defeitos estão citados abaixo como fatores que possam ter influenciado no resultado final.

> Vazamento do prompt, entidades duplicadas, loop no gleaning, erros que chegam à resposta, vazamento do prompt no corpus final, resultados que mudam entre execuções e recusa fixa em inglês.

Uso de IA no trabalho

O assistente Claude Code foi usado ao longo do trabalho como apoio. Pontos a relatar:

- **Onde ajudou:** organização do plano de execução do experimento; leitura de logs e do cache do GraphRAG para diagnosticar as falhas de integração da seção anterior; ajuste do settings.yaml e dos prompts.
- **Onde falhou ou precisou de correção:** Na organização do plano de execução, pois não levou em conta o tempo de indexação de documentos longos e a ordem lógica de implementação; Ao diagnosticar falhas, relatava problemas que já haviam sido resolvidos ou não tinham relação com os testes e sim com o ambiente.

- **O que foi verificado sem a IA:** Funcionamento do código, testes locais quanto ao tempo de execução e otimizações para o código, bugs no site usado para visualização do grafo e experimentação, qualidade das respostas, qualidade do grafo montado, critérios quanto usabilidade da ferramenta, etc.

5. Perspectivas futuras e mercado

Este bloco parte das limitações medidas no experimento para propor melhorias com base em algoritmos clássicos de grafos e em heurísticas, e depois discute tendências e adoção no mercado.

5.1 Proposta: algoritmos de grafos aplicados ao GraphRAG

O GraphRAG monta um grafo de conhecimento, mas usa muito pouco da teoria de grafos: o algoritmo de Leiden para formar comunidades e, na busca Local, apenas os vizinhos diretos das entidades da pergunta (um salto). Todo o resto do trabalho é feito pelo LLM, que é justamente a parte cara.

O diferencial desta proposta é tratar o grafo como uma estrutura de dados de verdade, e não só como uma lista de entidades. O grafo extraído tem pesos nas arestas (a força da relação, somada quando a relação aparece em mais de um chunk), é tratado como não direcionado, é desconexo (19 componentes no corpus pequeno) e tem ciclos (cerca de 82 ciclos independentes). Essas propriedades permitem aplicar algoritmos polinomiais, que rodam em milissegundos e não chamam o LLM, para decidir o que o LLM precisa ler. Cada proposta abaixo responde a um problema observado no experimento.

Algoritmos e o problema que cada um ataca

Algoritmo	Problema observado no experimento	Como ajudaria
Caminho mínimo (Dijkstra, com custo = 1 / força)	A busca Local só olha um salto e não superou o RAG vetorial nas perguntas multi-hop (1,50 contra 1,75)	Liga as entidades da pergunta por uma cadeia de relações e entrega ao LLM só o caminho e os chunks de origem, como uma cadeia de evidências. Na P05, o caminho ClickHouse → arquivo de features → Bot Management → limite de 200 features → FL2 → erro 5xx é a própria resposta
Árvore de Steiner (variante PCST)	Perguntas que citam três ou mais entidades, como a P04 (Delta, CrowdStrike, sistema de tripulação e processo)	Encontra o menor subgrafo que liga todas as entidades da pergunta. A aproximação clássica usa justamente uma árvore geradora mínima sobre as distâncias entre essas entidades
Árvore geradora máxima por comunidade (Kruskal ou Prim)	Relatórios de comunidade caros (48% da indexação) e que misturaram os dois incidentes	Mantém só as relações mais fortes de cada comunidade, formando um esqueleto do tópico. No corpus pequeno, a floresta geradora teria 143 arestas, contra 225 do grafo: 36% menos relações

Algoritmo	Problema observado no experimento	Como ajudaria
		para o LLM resumir
Centralidade de intermediação	Perguntas comparativas e globais, como a P08	Identifica as entidades-ponte entre comunidades, como "arquivo de configuração" e "atualização", que ligam os dois incidentes
Componentes conexas e Union-Find	Entidades duplicadas e três comunidades inteiras vindas dos exemplos do prompt	Une nomes parecidos num só nó e descarta componentes cujas entidades não aparecem no texto original
Decomposição k-core	Periferia do grafo com entidades soltas e pouco úteis	Poda os nós de grau baixo antes dos relatórios, reduzindo o que o LLM precisa processar

Heurísticas

Heurísticas são regras que trocam a garantia de resultado ótimo por velocidade. No GraphRAG, elas são a forma mais usada de reduzir o tempo, porque atacam diretamente as chamadas ao LLM:

- **Extração sem LLM.** Em vez de pedir ao LLM as entidades de cada chunk, extrair substantivos e expressões com processamento de linguagem natural e ligar os que aparecem juntos. O GraphRAG já tem esse modo (extract_graph_nlp, também chamado FastGraphRAG), mas a configuração padrão usa um extrator para inglês, inadequado para o corpus em português.
- **Menos passadas de extração.** O gleaning é uma passada extra por chunk. Com max_gleanings igual a 0, as 26 chamadas de extração do corpus pequeno cairiam para 13, ao custo de perder entidades que a primeira passada esquece.
- **Resumos sob demanda.** Em vez de escrever todos os relatórios de comunidade na indexação, escrevê-los só quando uma consulta precisar deles. É a ideia do LazyGraphRAG, da Microsoft [16].
- **Limiares e top-k.** Ler só as k comunidades ou os k caminhos mais relevantes, ou descartar relações com força abaixo de um limiar.
- **Busca A* e busca em feixe.** Para grafos grandes, o A* com uma heurística semântica (a distância entre o embedding de um nó e o da entidade de destino) e a busca em feixe, que guarda só os melhores caminhos parciais a cada passo, como no Think-on-Graph [15], exploram uma fração do grafo. No grafo deste trabalho, com 162 entidades, o Dijkstra exato já roda em menos de um milissegundo; a vantagem do A* só aparece em grafos com milhões de nós.

O preço das heurísticas é a qualidade. Elas podem deixar de fora a comunidade ou o caminho que continha a resposta, e por isso cada uma precisa ser medida com o mesmo benchmark antes de ser adotada.

Estimativa hipotética de ganho

As estimativas abaixo não foram medidas. Elas partem dos tempos reais do corpus pequeno (seção 3.2) e de hipóteses explícitas, para dar ordem de grandeza ao ganho possível. Indexação do corpus pequeno:

Cenário de indexação	Hipótese	Tempo estimado
Atual (medido)	Extração 809 s, relatórios 774 s, embeddings 19 s	1.603 s (26,7 min)
Moderado: + sem gleaning	Extração com metade das chamadas (809 s → cerca de 420 s)	≈ 1.010 s (−37%)
Agressivo: + resumos sob demanda	Relatórios saem da indexação e passam a ser gerados só quando uma consulta precisar	≈ 440 s (−73%), parte do custo vai para a consulta

Consulta, com as médias medidas na seção 3.2. Só a importação da biblioteca GraphRAG leva cerca de 8 a 9 segundos em cada consulta, porque o engine.py chama a linha de comando; um processo persistente, que mantém o índice carregado, eliminaria esse tempo:

Método de consulta	Hipótese	Média estimada
Basic (medido: 20,2 s)	Processo persistente, sem os 8 a 9 s de inicialização	≈ 11 s (−45%)
Local (medido: 20,8 s)	Processo persistente e contexto por caminho mínimo, menor que a vizinhança completa	≈ 10 a 11 s (−50%)
Global (medido: 44,9 s)	Pela P10, em que a etapa reduce não rodou, o map sobre os 36 relatórios custa cerca de 15 s e o reduce cerca de 20 s. Mapear só 25% das comunidades reduz o map para cerca de 4 s e o reduce para cerca de 15 s	≈ 28 s (−38%); ≈ 19 s (−58%) também com processo persistente

Para a qualidade, a hipótese principal é que a busca por caminhos aproxime a busca Local do RAG vetorial nas perguntas multi-hop, ou a supere: de 1,50 para pelo menos 1,75, a nota do Basic.

5.2 Tendências

A tendência mais clara é tornar o GraphRAG mais barato e mais preciso sem abrir mão do grafo. A própria Microsoft passou a adiar a escrita dos resumos de comunidade para a hora da consulta, no LazyGraphRAG [16], e a escolher dinamicamente quais comunidades a busca Global lê [17]. Na pesquisa, os trabalhos recentes recuperam caminhos e subgrafos em vez de vizinhanças inteiras [12], [13], [14], [15], que é a mesma direção da proposta da seção 5.1.

Outra tendência é usar o grafo como memória compartilhada entre agentes de IA. Antes de recuperar documentos, o sistema percorre o grafo e descobre relações indiretas entre as informações, entregando ao modelo a cadeia de contexto da pergunta, e não só um trecho parecido. Isso combina com

arquiteturas em que agentes acessam os sistemas de uma empresa por protocolos padronizados, como o Model Context Protocol (MCP), e precisam de um conhecimento consistente entre si.

Por fim, o grafo torna o conhecimento visível: quem usa a ferramenta consegue inspecionar como as informações se conectam, e a teoria de grafos abre espaço para otimizar tarefas complexas. O experimento deste trabalho mostra o outro lado dessa tendência: essas vantagens dependem de um grafo de boa qualidade, que ainda exige modelos maiores ou mais tempo de indexação.

5.3 Adoção na indústria

Os grandes provedores de nuvem e de bancos de grafos já oferecem GraphRAG como produto. A Microsoft mantém a biblioteca de código aberto usada neste trabalho [3] e integrou o LazyGraphRAG a produtos como o Microsoft Discovery [16]. A Amazon Web Services oferece GraphRAG no Amazon Bedrock Knowledge Bases, com o grafo guardado no Amazon Neptune Analytics [18]. A Neo4j mantém uma biblioteca oficial em Python para construir aplicações GraphRAG sobre o seu banco de grafos [19].

As aplicações mais promissoras são as bases de conhecimento grandes e cheias de relações entre documentos:

- Centrais de atendimento com milhares de artigos técnicos.
- Departamentos jurídicos que relacionam contratos, clientes e processos.
- Hospitais, conectando prontuários, exames e protocolos clínicos.
- Instituições financeiras, relacionando clientes, produtos, riscos e regulamentações.
- Indústrias, integrando manuais técnicos, manutenção e cadeia de fornecedores.
- Equipes de software, ligando registros de decisão de arquitetura, postmortems, atas e requisitos, como no cenário que motivou este trabalho.

Quanto mais complexa e interligada a base, maior o ganho potencial, desde que o grafo seja confiável. A demanda de mercado é por profissionais que combinem engenharia de dados, modelagem de grafos e integração de LLMs, porque o resultado depende menos de escrever código do zero e mais de configurar, avaliar e corrigir o pipeline, como mostrou o relato de experiência deste trabalho.

6. Conclusão e referências

6.1 Síntese crítica

O experimento respondeu à pergunta do trabalho de forma clara, mas contrária à expectativa inicial. Nos dois corpora, o RAG vetorial deu as melhores respostas e com o menor custo, e o GraphRAG se mostrou pior. A busca Local ficou próxima do RAG vetorial no corpus pequeno e caiu no corpus grande; a busca Global foi a mais lenta e a que mais misturou fatos; e a indexação do grafo custou muito mais, crescendo mais rápido que o próprio texto. O único sinal a favor do grafo apareceu nas perguntas multi-hop do corpus grande, em que a busca Local resistiu melhor aos distratores do que o RAG vetorial.

Os fatores que podem explicar esse resultado são:

- **LLM com poucos parâmetros.** O modelo de 4B extraiu um grafo ruidoso, com entidades copiadas dos exemplos do prompt, nomes duplicados e fatos atribuídos ao incidente errado.

Como o grafo é a base das buscas Local e Global, um erro de indexação vira erro de resposta. Um modelo maior tende a extrair um grafo melhor, mas exige mais memória e mais tempo.

- **Configurações não exploradas.** Mais passadas de extração, chunks menores, mais tipos de entidade e o ajuste automático dos prompts de extração para o domínio, que o GraphRAG oferece, poderiam melhorar o grafo. Todas elas aumentam o tempo de indexação, que já era o maior custo do GraphRAG.
- **Uso limitado do grafo pela biblioteca.** O GraphRAG usa do grafo só o agrupamento em comunidades e a vizinhança direta das entidades. Os algoritmos propostos na seção 5.1, como caminhos mínimos, árvore de Steiner, árvore geradora máxima e a limpeza por componentes, atacam exatamente as falhas observadas e poderiam melhorar a recuperação e reduzir o custo.
- **Limites do próprio experimento.** Poucas perguntas, um único modelo, uma única execução por método, avaliação manual feita por uma pessoa (sem cegueira completa na segunda rodada) e uma configuração de concorrência diferente entre os dois corpora. As conclusões valem para este cenário e não devem ser generalizadas sem novos testes.

Desses resultados sai a regra de decisão proposta na introdução: use o RAG vetorial como padrão, porque é mais barato, mais rápido e foi mais preciso; considere o GraphRAG quando a base for grande e muito interligada, quando as perguntas exigirem ligar fatos de documentos diferentes ou ter uma visão do conjunto, quando houver um modelo de linguagem capaz de extrair um grafo confiável e quando o corpus mudar pouco, para que o custo de reindexar compense.

O veredito é que, para o cenário estudado (execução local, modelo pequeno e documentos técnicos), o GraphRAG é hoje um nicho experimental, e não uma aposta segura. A ideia é sólida e o mercado investe nela, mas o ganho depende da qualidade do grafo, e essa qualidade ainda custa caro. Os caminhos mais promissores são os que reduzem esse custo ou aproveitam melhor o grafo, como os resumos sob demanda e os algoritmos de grafos discutidos no bloco 5.

Como aprendizado de Engenharia de Software, o trabalho mostrou o valor de medir antes de adotar: uma técnica nova, bem documentada e promovida por uma grande empresa perdeu para uma solução mais simples no cenário real testado. Sem um benchmark com perguntas e gabarito definidos antes, essa conclusão teria sido trocada pelo discurso de divulgação da tecnologia.

6.2 Referências

[1] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, 2020. Disponível em: https://arxiv.org/abs/2005.11401

[2] D. Edge et al., "From Local to Global: A Graph RAG Approach to Query-Focused Summarization," arXiv:2404.16130, 2024. Disponível em: https://arxiv.org/abs/2404.16130

[3] Microsoft, "GraphRAG: documentação oficial." Disponível em: https://microsoft.github.io/graphrag/

[4] V. A. Traag, L. Waltman e N. J. van Eck, "From Louvain to Leiden: guaranteeing well-connected communities," Scientific Reports, vol. 9, art. 5233, 2019.

[5] A. Vaswani et al., "Attention Is All You Need," in Advances in Neural Information Processing Systems (NeurIPS), vol. 30, 2017.

[6] Z. Ji et al., "Survey of Hallucination in Natural Language Generation," ACM Computing Surveys, vol. 55, n. 12, 2023.

[7] V. Karpukhin et al., "Dense Passage Retrieval for Open-Domain Question Answering," in Proc. EMNLP, 2020.

[8] A. Hogan et al., "Knowledge Graphs," ACM Computing Surveys, vol. 54, n. 4, 2021.

[9] Wikipedia, "2024 CrowdStrike-related IT outages", revisão 1376517477, 24 set. 2026. Disponível em: https://en.wikipedia.org/wiki/2024_CrowdStrike-related_IT_outages. Licença CC BY-SA 4.0.

[10] M. Prince, "Cloudflare outage on November 18, 2025," The Cloudflare Blog, 2025. Disponível em: https://blog.cloudflare.com/18-november-2025-outage/

[11] Wikipedia em português, artigos "Incidente da CrowdStrike em 2024", "CrowdStrike", "Microsoft Azure", "Rede de fornecimento de conteúdo", "Proxy reverso", "Ataque de negação de serviço", "Interrupção do Facebook de 2021", "Log4Shell", "WannaCry", "Voo Ariane V88", "Therac-25" e "Bug do milênio", acesso em 24 out. 2026. Disponível em: https://pt.wikipedia.org/. Licença CC BY-SA 4.0. Revisões usadas em docs/corpus_fontes.md.

[12] B. Chen et al., "PathRAG: Pruning Graph-based Retrieval Augmented Generation with Relational Paths," in Proc. AAAI Conference on Artificial Intelligence, vol. 40, 2026. Disponível em: https://arxiv.org/abs/2502.14902

[13] X. He et al., "G-Retriever: Retrieval-Augmented Generation for Textual Graph Understanding and Question Answering," in Advances in Neural Information Processing Systems (NeurIPS), 2024. Disponível em: https://arxiv.org/abs/2402.07630

[14] B. J. Gutiérrez et al., "HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models," in Advances in Neural Information Processing Systems (NeurIPS), 2024.

[15] J. Sun et al., "Think-on-Graph: Deep and Responsible Reasoning of Large Language Model on Knowledge Graph," in International Conference on Learning Representations (ICLR), 2024. Disponível em: https://arxiv.org/abs/2307.07697

[16] Microsoft Research, "LazyGraphRAG: Setting a new standard for quality and cost," Microsoft Research Blog, 25 nov. 2024. Disponível em: https://www.microsoft.com/en-us/research/blog/lazygraphrag-setting-a-new-standard-for-quality-and-cost/

[17] Microsoft Research, "GraphRAG: Improving global search via dynamic community selection," Microsoft Research Blog, 2024. Disponível em: https://www.microsoft.com/en-us/research/blog/graphrag-improving-global-search-via-dynamic-community-selection/

[18] Amazon Web Services, "Amazon Bedrock Knowledge Bases supports GraphRAG now generally available," mar. 2025. Disponível em: https://aws.amazon.com/about-aws/whats-new/2025/03/amazon-bedrock-knowledge-bases-graphrag-generally-available

[19] Neo4j, "Neo4j GraphRAG for Python" (biblioteca neo4j-graphrag). Disponível em: https://neo4j.com/docs/neo4j-graphrag-python/